Floating Point

Floating Point

- Representation for non-integral numbers
  - Including very small and very large numbers
- Like scientific notation
  - -2.34×10^{56}
  - $+0.002 \times 10^{-4}$
  - $+987.02 \times 10^{9}$

normalized

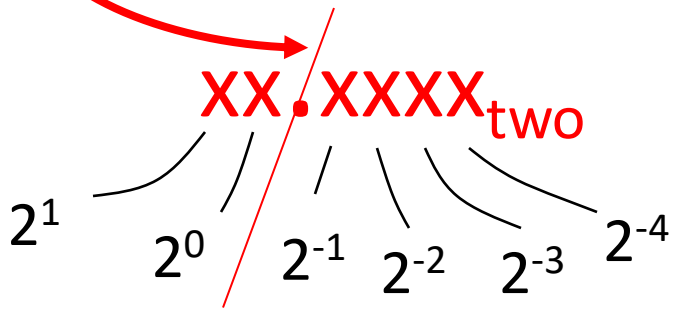
not normalized

- In binary
  - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$

Representation of Fractions

- Introduce a fixed "Binary Point" that signifies boundary between negative & nonnegative powers

Example 6-bit representation:



- Example
  - $10.1010_{two} = 1\times2^1 + 1\times2^{-1} + 1\times2^{-3} = 2.625_{ten}$

Scientific Notation (Decimal)

mantissa

Exponent

$$5.566 \times 10^{1}$$

decimal point

radix (base)

- Representation of floating point number is not unique
  - 5.566×10^{1}, 0.5566×10^{2}, 0.05566×10^{3}
- Normalized form: exactly one digit (non-zero) to left of decimal point
- Alternatives to representing 1/1,000,000,000
  - Normalized: 1.0×10^{-9}
  - Not normalized: 0.1×10^{-8}, 10.0×10^{-10}

Scientific Notation (Binary)

mantissa

exponent

$$1.0101_{two} \times 2^{1}$$

binary point

radix (base)

- Computer arithmetic that supports this called floating point due to the "floating" of the binary point
  - Declare such variable in C as `float`

Translating To and From Scientific Notation

- Consider the number $1.011_{two} \times 2^4$
- To convert to ordinary number, shift the decimal to the right by 4
  - Result: $10110_{two} = 22_{ten}$
- For negative exponents, shift decimal to the left
  - $1.011_{two} \times 2^{-2} \Rightarrow 0.01011_{two} = 0.34375_{ten}$
- Go from ordinary number to scientific notation by shifting until in *normalized* form
  - $1101.001_{two} \Rightarrow 1.101001_{two} \times 2^3$

Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
  - Portability issues for scientific code
- Now almost universally adopted
- Two representations
  - Single precision (32-bit)
  - Double precision (64-bit)

Floating Point Encoding

- Use normalized, Base 2 scientific notation

$$+1.xxx\cdots x_{two} \times 2^{yyy\cdots y}$$

- Split a 32-bit word into 3 fields

  - S represents Sign (1 is negative, 0 positive)
  - Exponent represents y's
  - Mantissa represents x's

- How about 64 bit?

The Exponent Field

- Use biased notation
  - Read exponent as unsigned, but with *bias* of 127
  - Exp. field ranges from -127($0000\ 0000_{two}$) to 128($1111\ 1111_{two}$)
  - Exponent 0 is represent as $0111\ 1111_{two} = 127_{ten}$
- To encode in biased notation, add the bias (add 127) then encode in unsigned:
  - If we had 2^1, exp = 1 → 128 → $1000\ 0000_{two}$
  - 2^{127}: exp = 127 → 254 → $1111\ 1110_{two}$

The Exponent Field

- Why use biased notation for the exponent?

$0.5_{ten} \rightarrow 1.0 \times 2^{-1}$

$1.0_{ten} \rightarrow 1.0 \times 2^{1}$

	Two's complement
-1	1111 1111
+1	0000 0001

  - Remember that we want floating point numbers to look small when their actual value is small
    - We don't like how in 2's complement, -1 looks bigger than 0
- To avoid this, biased notation is used

-1 is represented as $-1 + 127 = 126_{ten} = 0111\ 1110_{two}$

0 is represented as $0 + 127 = 127_{ten} = 0111\ 1111_{two}$

$+1$ is represented as $+1 + 127 = 128_{ten} = 1000\ 0000_{two}$

Floating Point Encoding

31	30 … 23	22 … 0
S	Exponent	Mantissa
1 bit	8 bits	23 bits

$$(-1)^{S} \times (1\ .\ \text{Mantissa}) \times 2^{(\text{Exponent}-127)}$$

- $2003.0 = 0111\ 1101\ 0011_{two}$
  - ➔ $(-1)^{0} \times 1.1111010011 \times 2^{10}$
  - S = 0
  - Exp = E + bias = 10 + 127 = 137 = $1000\ 1001_{two}$

$2003.0 = 0100\ 0100\ 1111\ 1010\ 0110\ 0000\ 0000\ 0000_{two}$

$= 0x44FA\ 6000$

Precision and Accuracy

- Precision is a count of the number of bits in a computer word used to represent a value
- Accuracy is a measure of the difference between the actual value of a number and its computer representation
- Higher precision permits high accuracy but doesn't guarantee it. It is possible to have high precision, but low accuracy.
- Example: `float pi = 3.14;`
  - `pi` will be represented using all 24 bits of the significant (highly precise), but is only an approximation (not accurate)

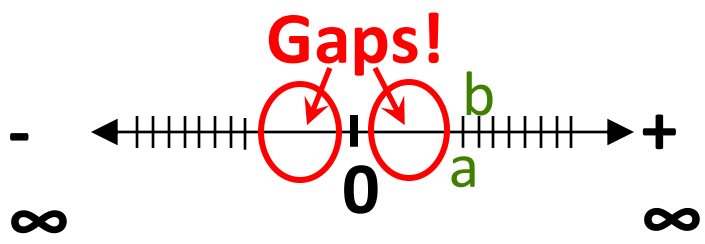
Special Case #1: Zero

- But wait… what happened to zero?
  - Using standard encoding 0x00000000 is $1.0 \times 2^{-127} \neq 0$

  - *Special case:* Exp and Man all zeros = 0

Special Case #2: Very Small Numbers

- What are the normal numbers closest to 0?
  (Here, normal means the exponent is **nonzero**)
  - $a = 1.0\cdots00_{two} \times 2^{1-127} = (1+0) \times 2^{-126} = 2^{-126}$
  - $b = 1.0\cdots01_{two} \times 2^{1-127} = (1+2^{-23}) \times 2^{-126} = 2^{-126} + 2^{-149}$



- The gap between 0 and a is 2^{-126}
  - The gap between a and b is 2^{-149}
- *Special case:* Exp =0 and Man ≠ 0 mean denorm numbers

Denorm Numbers

- Short for "denormalized numbers"
  - No leading 1
  - Careful! Denorm exponent bias is now 126 when Exp = 0x00 (intuitive reason: the floating point moves one more bit to the left of the leading bit)
- Now what do the gaps look like?
  - Smallest denorm: $\pm\ 0.0\cdots01_{two}\times2^{-126} = \pm\ 2^{-149}$
  - Smallest norm: $\pm\ 1.0\cdots0_{two}\times2^{-126} = \pm\ 2^{-126}$

So much closer to 0

Other Special Cases

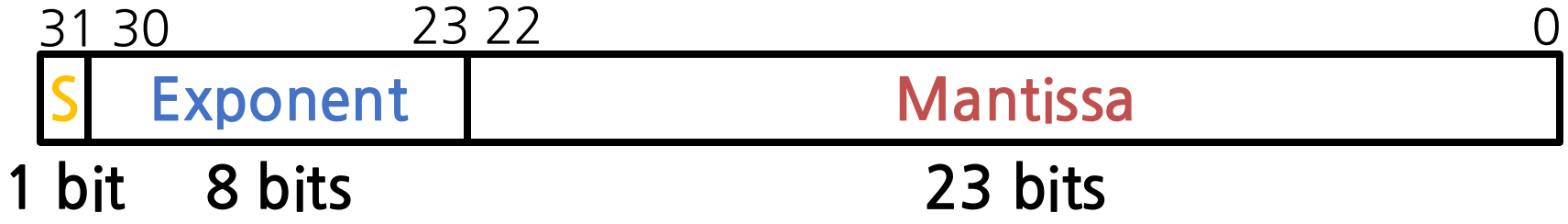
- Exp = 0xFF, Man = 0: ±∞
  - e.g. division by zero

- Exp = 0xFF, Man ≠ 0: Not a Number (NaN)
  - e.g. square root of negative number, 0/0, ∞-∞

- https://en.wikipedia.org/wiki/IEEE_754

Floating Point Numbers Summary

Exponent	Mantissa	Meaning
0x00	0	± 0
0x00	non-zero	± Denorm Num
0x01 - 0xFE	anything	± Norm Num
0xFF	0	± ∞
0xFF	non-zero	NaN



Example: Convert FP to Decimal

0	0110 1000	101 0101 0100 0011 0100 0010

- Sign: 0 means positive
- Exponent:
  - $0110\ 1000_{two} = 104_{ten}$
  - Bias adjustment: 104 - 127 = -23
- Mantissa:

  1 . 10101010100001101000010

  $= 1 + 1x2^{-1} + 0x2^{-2} + 1x2^{-3} + 0x2^{-4} + 1x2^{-5} + \ldots$

  $= 1 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-9} + 2^{-14} + 2^{-15} + 2^{-17} + 2^{-22}$

  $= 1.0 + 0.666115$
- Represents: $1.666115_{ten} \times 2^{-23} \approx 1.986 * 10^{-7}$

Example: Convert to FP

1/3

$= 0.33333\cdots_{ten}$

$= 0.25 + 0.0625 + 0.015625 + 0.00390625 + \cdots$

$= 1/4 + 1/16 + 1/64 + 1/256 + \cdots$

$= 2^{-2} + 2^{-4} + 2^{-6} + 2^{-8} + \cdots$

$= 0.0101010101\cdots_{two} \times 2^{0}$

$= 1.0101010101\cdots_{two} \times 2^{-2}$

- Sign: 0
- Exponent = -2 + 127 = 125 = 01111101_{two}
- Mantissa = $0101010101\cdots_{two}$

0	0111 1101	010 1010 1010 1010 1010 1010

FP Adder Hardware

- Much more complex than integer adder
- Doing it in one clock cycle would take too long
  - Much longer than integer operations
  - Slower clock would penalize all instructions
- FP adder usually takes several cycles
  - Can be pipelined

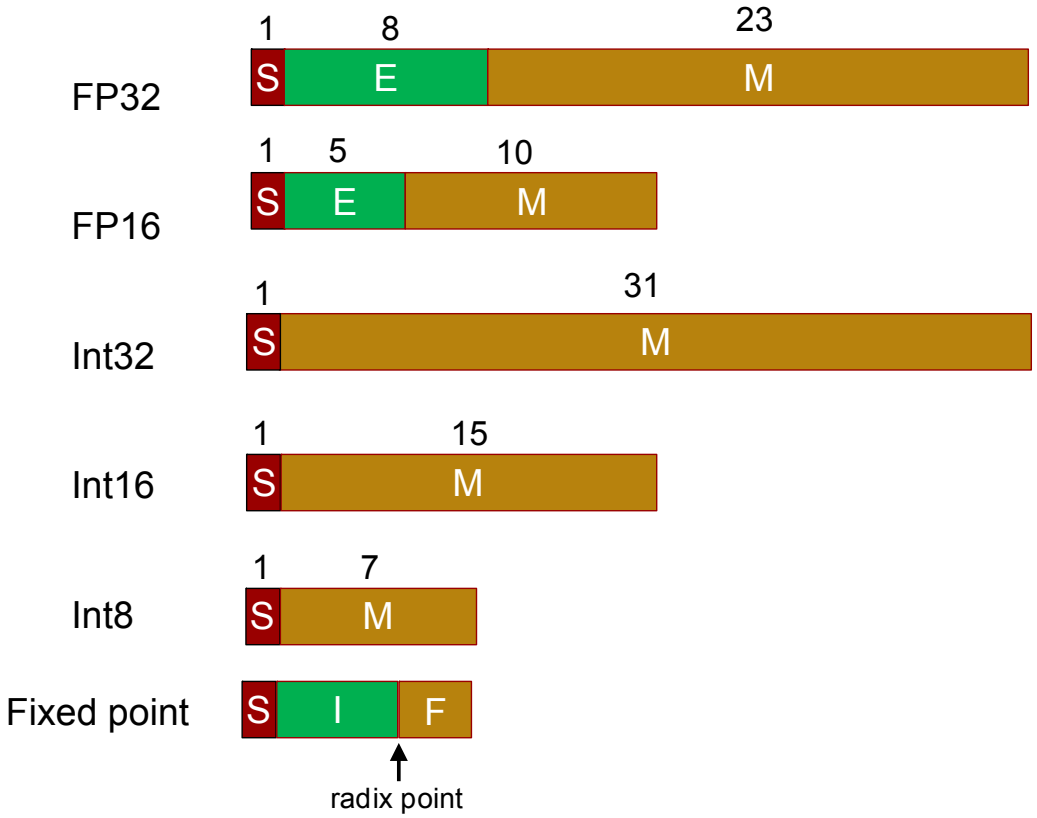
Summary

- Bits have no inherent meaning
  - Interpretation depends on the instructions applied
- Computer representations of numbers
  - Finite range and precision
  - Need to account for this in programs
- ISAs support arithmetic
  - Signed and unsigned integers
  - Floating-point approximation to reals
- Bounded range and precision
  - Operations can overflow and underflow

Extra slides

Hardware 101: Number Representation

$$(-1)^S \times (1.M) \times 2^E$$

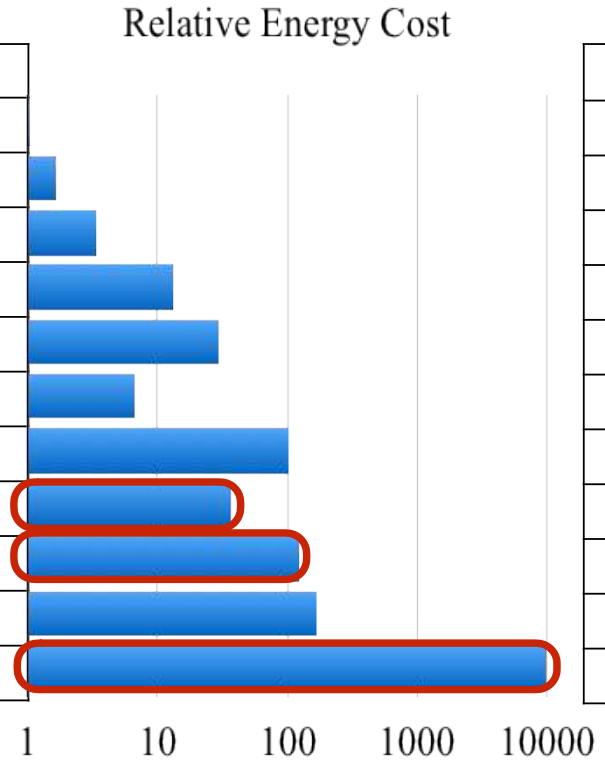


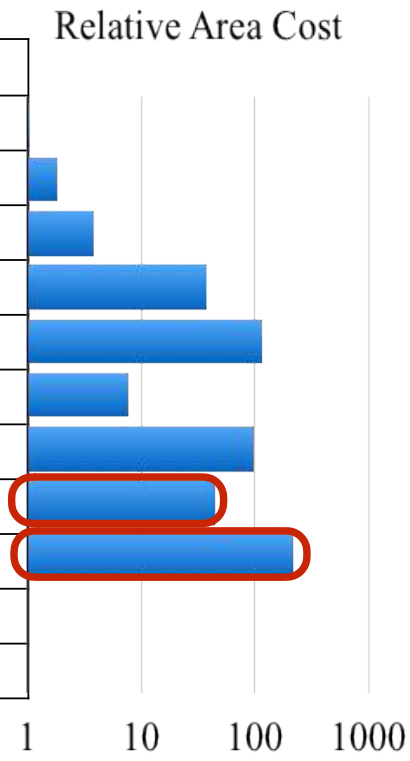
	Range	Accuracy
FP32	10^{-38} - 10^{38}	.000006%
FP16	$6x10^{-5}$ - $6x10^{4}$	.05%
Int32	0 – $2x10^{9}$	½
Int16	0 – $6x10^{4}$	½
Int8	0 – 127	½
Fixed point	-	-

Dally, High Performance Hardware for Machine Learning, NIPS'2015

Hardware 101: Number Representation

Operation:	Energy (pJ)	Area (μm²)
8b Add	0.03	36
16b Add	0.05	67
32b Add	0.1	137
16b FP Add	0.4	1360
32b FP Add	0.9	4184
8b Mult	0.2	282
32b Mult	3.1	3495
16b FP Mult	1.1	1640
32b FP Mult	3.7	7700
32b SRAM Read (8KB)	5	N/A
32b DRAM Read	640	N/A





Energy numbers are from Mark Horowitz "Computing's Energy Problem (and what we can do about it)", ISSCC 2014
Area numbers are from synthesized result using Design Compiler under TSMC 45nm tech node. FP units used DesignWare Library.